

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	P. Lohith Krishna	Reg. No.:	192325122
Date:	17.08.2026	Duration:	60 minutes

Code of Conduct

I P. Lohith Krishna 192325122 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action. Signature of the Student: P. Lohith Krishna

Annexure A – Git & Docker Technical Assignment

Smart Campus Maintenance and Service Request Management System

1. Problem Analysis

1.1 Introduction

The Smart Campus Maintenance and Service Request Management System is a web-based application designed to help students, faculty, and staff report campus service issues and track their resolution efficiently. Manual reporting through phone calls, messages, or paper forms can be slow and may make it difficult to track the status, priority, and responsibility of each request.

The proposed system provides a centralized platform where users can submit service requests, select issue categories, describe problems, attach relevant details, track request status, and receive updates. Administrators can assign requests to maintenance staff, monitor progress, manage categories, and generate service reports.

The application follows a client-server architecture where the user interacts with a web interface, the backend manages request processing and business rules, and the database stores users, service requests, assignments, status history, and notification information.

1.2 Problem Statement

Students and staff often face difficulties in reporting and following up on campus maintenance issues because requests may be submitted through different channels and there is no single place to track their progress. The proposed system aims to simplify this process by providing:

The proposed system aims to simplify this process by providing:

- Online service request submission.
- Issue categorization and priority management.
- Technician assignment and request tracking.
- Status updates and service history.
- Dashboard and service reports.
- Secure storage of user and request information.
- Easy access to service information through a web interface.

1.3 Objectives

The major objectives of the system are:

1. To allow users to submit campus maintenance and service requests online.
2. To reduce delays caused by manual reporting and follow-up.
3. To maintain organized records of users, requests, assignments, and status history.
4. To help administrators assign and monitor service requests efficiently.
5. To provide request status, service summaries, and performance reports.
6. To provide a simple and user-friendly interface.
7. To securely store user and service request information.
8. To develop a portable application using Docker.
9. To maintain source-code history using Git.
10. To provide a scalable architecture for future campus service enhancements.

1.4 Functional Requirements

User Registration and Login

The system shall allow students, faculty, staff, technicians, and administrators to create authorized accounts and securely log in.

Service Request Management

Users can create and manage service requests by providing details such as:

- Issue category
- Location or room

- Problem description
- Priority level
- Additional remarks

Request Tracking

Users can view their submitted requests and track the current status and service history.

Request Processing

The system processes service requests based on:

- Request category
- Reported location
- Priority level
- Assigned technician
- Current request status

Technician Assignment

Administrators can assign eligible technicians to requests and update assignments when required.

Notifications and Updates

The system provides service updates such as:

- Request submitted
- Technician assigned
- Work in progress
- Request completed
- Request reopened

Reports

The system generates summaries containing:

- Total requests
- Open and completed requests
- Requests by category
- Requests by priority
- Average resolution time
- Technician workload
- Service history information

Administrator Functions

An administrator can manage users, technicians, service categories, request priorities, assignments, and system information.

1.5 Non-Functional Requirements

Requirement	Description
Performance	Service requests should be processed and displayed quickly.
Security	User, request, and technician data should be protected.
Reliability	The system should maintain consistent request status and assignment records.
Scalability	The application should support increasing users and service requests.
Maintainability	Source code should be modular and easy to modify.
Portability	The application should run consistently using Docker.
Usability	The interface should be simple and understandable.
Availability	The application should be available whenever the service is running.

1.6 Expected Outcomes

The expected outcome is a web-based campus service management application capable of recording maintenance requests, assigning service staff, tracking progress, and generating useful service reports.

The system should maintain accurate request records, provide clear status information, support administrator monitoring, and make campus service coordination more organized and transparent.

2. Project Structure Design A suitable repository structure is:

smart-campus-service-management/

```
|
|
|— frontend/
|  |— public/
|  |— src/
|  |— components/
|  |— pages/
|
|— backend/
|  |— controllers/
```

```
|   ├── models/
|   ├── routes/
|   ├── services/
|   ├── middleware/
|   ├── config/
|   └── server.js
|
|── database/
|   └── seed/
|
|── tests/
|   ├── unit/
|   └── integration/
|
|── docs/
|   ├── architecture/
|   └── screenshots/
|
|── .gitignore
|── .env.example
|── Dockerfile
|── docker-compose.yml
|── package.json
|── README.md
└── LICENSE
```

2.1 Folder Description frontend/

Contains the user interface of the campus service management application.

backend/

Contains server-side application logic for request processing, assignment, authentication, and reporting.

controllers/

Processes requests and sends responses.

models/

Defines database models such as User, ServiceRequest, Category, Technician, StatusHistory, and Notification.

routes/

Defines API endpoints.

services/

Contains business logic such as service request processing, technician assignment, notification handling, and reporting.

middleware/

Contains authentication, authorization, validation, and error-handling middleware. database/

Contains database initialization and seed data.

tests/

Contains unit and integration tests. docs/

Contains architecture diagrams, screenshots, and project documentation.

Dockerfile

Contains instructions for building the application container image.

docker-compose.yml

Defines and manages multiple application services.

.gitignore

Specifies files and folders that Git should not track, such as:

node_modules/

.env logs/

uploads/

.env.example

Provides sample environment variables without exposing sensitive credentials.

README.md

Contains project description, installation instructions, Git workflow, Docker commands, and usage instructions.

3. Git Repository Initialization

Git is used to track changes made to the campus service management source code.

3.1 Repository Initialization

The following commands can be used:

```
mkdir smart-campus-service-  
management cd smart-campus-  
service-management
```

```
git init
```

```
git status
```

```
git add .
```

```
git commit -m "chore: initialize campus service repository"
```

The remote repository can then be connected:

```
git remote add origin <repository-url>
```

```
git branch -M main
```

```
git push -u origin main 3.2
```

Important Git Files .git/

The .git directory contains Git's internal repository information, including:

- Commit history
- Branch information
- Configuration
- Object database
- References

.gitignore

Prevents unnecessary or sensitive files from being committed.

README.md

Provides project documentation for developers and users.

4. Git Branching Strategy

A Git Flow-inspired branching strategy is suitable for this project.

main

|

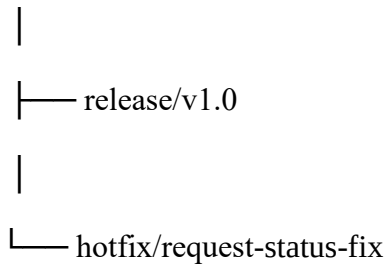
|— develop

| |— feature/user-authentication

| |— feature/service-request

| |— feature/technician-assignment

| |— feature/service-reports



4.1 Main Branch

The main branch contains stable production-ready code.

4.2 Develop Branch

The develop branch contains integrated development code before release.

4.3 Feature Branches

Individual features are developed independently.

Examples:

feature/user-authentication feature/service-

request feature/technician-assignment

feature/service-reports

4.4 Release Branch

A release branch is created when the application is ready for final testing.

Example: release/v1.0

4.5 Hotfix Branch

A hotfix branch is used to correct critical issues in the production version.

Example:

hotfix/request-status-error 4.6

Justification

This branching strategy is appropriate because the project contains several independent campus service modules.

Developers can work on different features without directly affecting the stable application.

It also provides:

- Better collaboration
- Controlled integration
- Easy bug fixing
- Stable production code
- Organized release management
- Clear development history

5. Version Control Workflow

The development workflow can be demonstrated using the following sequence.

5.1 Create Development Branch `git checkout -b develop`

`git push -u origin develop`

5.2 Create Feature Branch `git checkout develop`

`git checkout -b feature/service-request` After

implementing the service request feature:

`git status`

`git add .`

`git commit -m "feat: implement service request module"`

Push the feature branch:

`git push -u origin feature/service-request`

5.3 Merge

Feature After

testing:

`git checkout develop`

```
git pull origin develop
```

```
git merge feature/service-request
```

Push the updated development branch:

```
git push origin develop
```

5.4 Conflict

Resolution If Git

reports a conflict:

```
git status
```

Open the conflicting file and resolve the conflict.

After resolving: git

```
add .
```

```
git commit -m "fix: resolve service request merge conflict"
```

5.5 Synchroni

zation Before

starting new

work: git pull

origin develop

After completing

work: git push

origin develop

5.6 Release git

checkout develop

```
git checkout -b release/v1.0
```

git checkout main

git merge release/v1.0

git tag v1.0

git push origin main --tags

6. Commit History Analysis A meaningful commit history can be:

a81c123 feat: implement user authentication b92d452 feat: add service
request module c73e981 feat: implement technician assignment
d64f210 feat: add request tracking e51a321 feat: implement
notifications f42b876 feat: add service reports g39c542 test: add service
request tests h28b643 docker: add application Dockerfile i17a532
docker: add compose configuration j06b421 docs: update project
documentation

6.1 Commit Message Practices

The project follows meaningful commit messages using prefixes such as:

feat: fix:

test: docs:

refactor:

chore:

docker:

Examples:

```
git commit -m "feat: implement service request module"
```

```
git commit -m "fix: update request status validation"
```

```
git commit -m "test: add service request unit tests"
```

```
git commit -m "docker: add MongoDB service"
```

```
git commit -m "docs: update Docker deployment instructions"
```

These messages make the history easier to understand and maintain.

6.2 Viewing History git log

```
--oneline --graph --all
```

This command displays commits, branches, and merges in a compact graphical format.

Screenshot to include: Git log showing the complete commit history.

7. Docker Environment Design

Docker is suitable for the Smart Campus Maintenance and Service Request Management System because it provides a consistent execution environment.

Without Docker, developers may have different versions of:

- Node.js
- Database software
- Operating system libraries
- Application dependencies

Docker packages the application and its dependencies into containers.

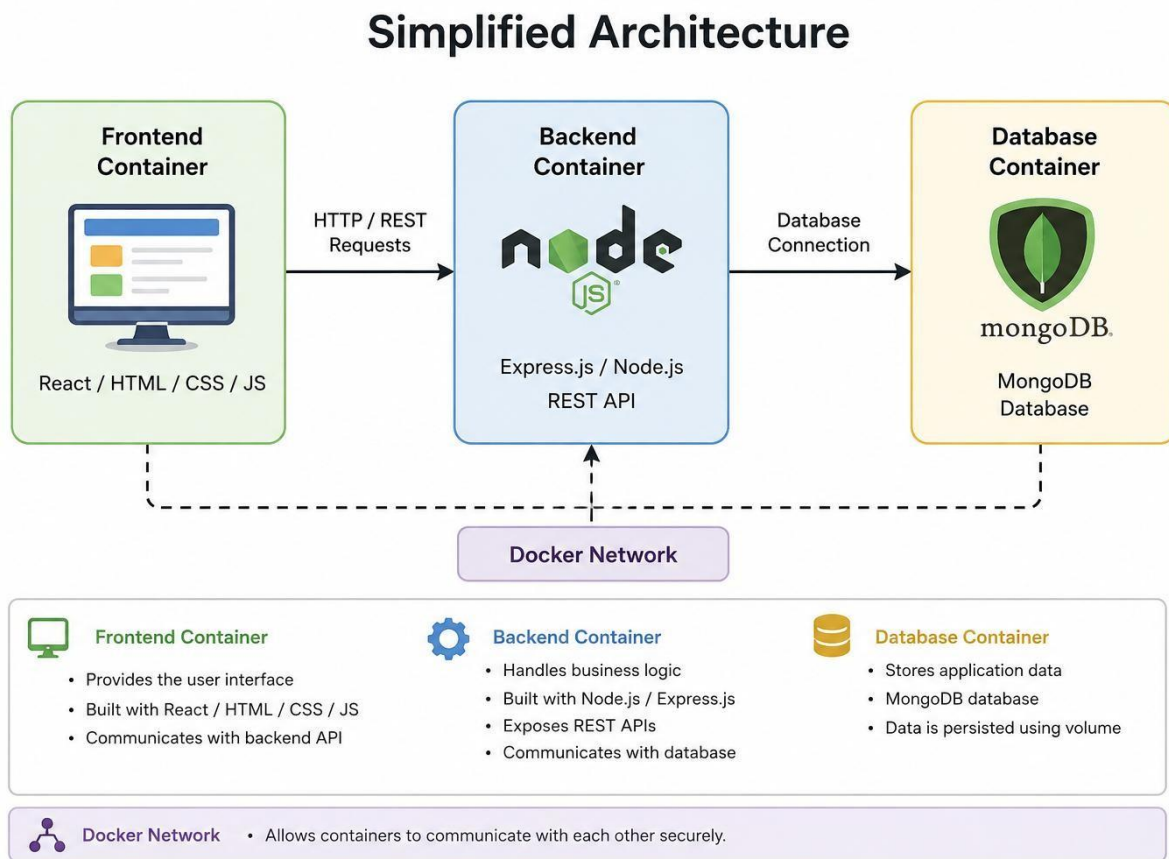
7.1 Components Requiring Containerization

The proposed application contains:

1. Frontend
2. Backend/API server

3. Database

A simplified architecture is:



8. Dockerfile Development

A Dockerfile can be created for the backend application.

```
FROM node:20-alpine
```

```
WORKDIR /app
```

```
COPY package*.json ./
```

```
RUN npm ci --omit=dev
```

COPY . .

EXPOSE 5000

CMD ["npm", "start"]

8.1 Dockerfile Explanation

FROM

FROM node:20-alpine

Provides Node.js and a lightweight Linux environment.

WORKDIR

WORKDIR /app

Sets /app as the working directory.

COPY

COPY package*.json ./

Copies package configuration files.

RUN

RUN npm ci --omit=dev

Installs production dependencies.

COPY

COPY . .

Copies application source code into the container.

EXPOSE

EXPOSE 5000

Documents that the application uses port 5000.

CMD

CMD ["npm", "start"]

Starts the application when the container runs.

9. Docker Compose Design

Docker Compose can be used to manage the backend and database together.

Example:

services:

 backend:

 build: .

 container_name: campus-service-backend

 ports:

 - "5000:5000" environment:

 PORT: 5000

 DATABASE_URL: mongodb://database:27017/campus_service

 depends_on:

 - database

 database:

 image: mongo:7 container_name:

 campus-service-database

 ports:

 - "27017:27017" volumes:

 - mongodb_data:/data/db

volumes:

mongodb_data:

9.1 Backend Service

The backend service builds the campus service application using the

Dockerfile. backend:

build: .

The application is exposed through:

localhost:5000

9.2 Database Service

The database uses the official MongoDB image. image:

mongo:7

The database is internally available to the backend through the service name:

database Therefore:

mongodb://database:27017/campus

_service is used instead of

localhost.

9.3 Networking

Docker Compose creates a network that allows the containers to communicate with each other. Backend

Container

|

| Docker Network

|

v

Database Container

9.4 Volumes

volumes:

mongodb_data:

The volume preserves database data even if the database container is recreated.

9.5 Environment Variables

Environment variables prevent sensitive configuration from being hard-coded.

Example:

PORT=5000

DATABASE_URL=mongodb://database:27017/campus_service

For production deployment, database credentials and secret keys should be stored securely rather than directly in the Compose file.

10. Container Deployment and Testing

10.1 Build and Start Containers

Run: docker compose up -

-build

The command builds the application image and starts all required services.

To run in detached mode: docker

compose up -d --build 10.2

Check Running Containers

docker ps

Expected output should show containers similar to:

campus-service-backend

campus-service-database 10.3

View Logs docker compose

logs backend The application

should display a successful
server-start message.

10.4 Test the Application Open

the application in a browser:

`http://localhost:5000`

The following functions should be tested:

1. User registration and login
 2. Service request submission
 3. Request category and location entry
 4. Priority selection
 5. Technician assignment
 6. Request status update
 7. Notification/update handling
 8. Service report generation
 9. Database storage
- #### 10.5 Container Testing
- Check the containers: `docker compose ps` Check

database logs:

`docker compose logs database`

Stop the application: `docker`

`compose down`

Start it again:

`docker compose up -d`

The database information should remain available because of the Docker volume.

Required Screenshots

Include screenshots of:

- `git init`
- `git status`
- Git branches
- Feature branch creation
- `Git commit`

- `Git merge`

- `git log --oneline --graph --all`
 - `docker build`
 - `docker compose up`
 - `docker ps`
 - Docker logs
 - Application running in browser
 - Database/container status
-

11. Benefits of Git and Docker

11.1 Git Benefits

Collaboration

Multiple developers can work on separate branches.

Version Management

Git maintains the complete history of source-code changes.

Branching

Features can be developed independently.

Rollback

Previous versions can be restored when necessary.

Code Review

Feature branches and pull requests allow changes to be reviewed before integration.

Maintainability

Meaningful commits make it easier to identify when and why changes were made.

11.2 Docker Benefits

Portability

The application can run consistently on different systems.

Dependency Management

Application dependencies are packaged within the container.

Deployment

The application can be deployed using the same container configuration used during development.

Isolation

Application and database services are isolated into separate containers.

Scalability

Additional application containers can be created when required.

Reproducibility

Developers can recreate the same environment using the Dockerfile and Compose configuration.

12. Challenges and Improvements

12.1 Challenges

Git Merge Conflicts

Changes made by different developers can cause conflicts.

Dependency Compatibility

Different package versions can sometimes create compatibility problems.

Docker Networking

The backend must use the database service name instead of localhost when communicating between containers.

Environment Configuration

Database credentials and secret keys must be configured correctly.

Database Persistence

Without volumes, database data may be lost when containers are removed.

Service Policy Changes

Campus service categories, priorities, or workflows may change over time, requiring the application logic and configuration to be updated.

12.2 Improvements

Future versions can include:

- Automated CI/CD using GitHub Actions.
- Automated unit and integration testing.
- Multi-stage Docker builds.
- Docker image optimization.
- HTTPS support.
- Role-based access control.
- Two-factor authentication.
- Automated database backups.
- Service-category and workflow configuration through an administrator dashboard.
- AI-assisted service request classification and priority recommendations.
- Maintenance schedule and preventive service tracking.
- Technician workload and performance analysis.
- Mobile application support.
- Cloud deployment.
- Monitoring and logging.
- Automated security scanning.

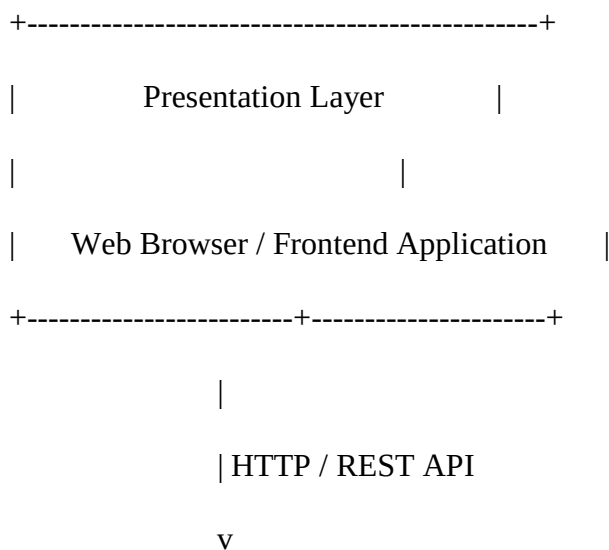
13. Software Architecture Selection

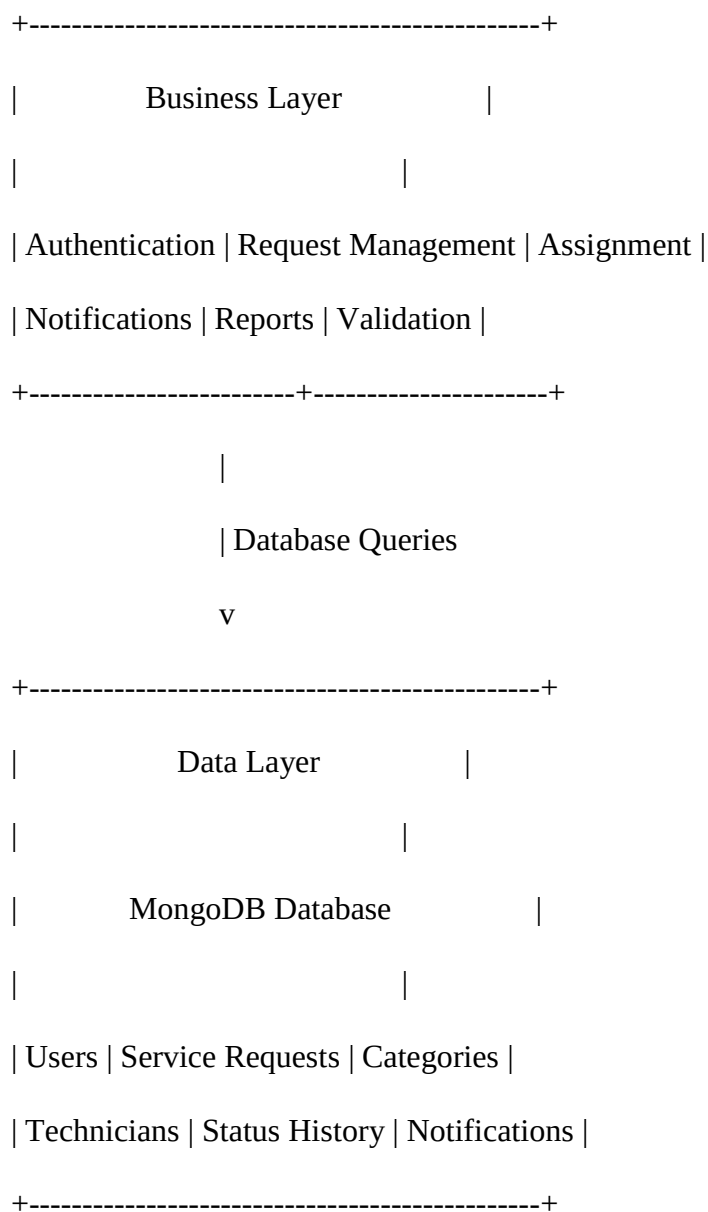
13.1 Selected Architecture

The proposed system uses a Three-Tier Architecture consisting of:

1. Presentation Layer
2. Business Logic Layer
3. Data Layer

This architecture separates the user interface, application logic, and database operations.





13.2 Presentation Layer

The presentation layer provides the user interface.

Main screens may include:

- Login
- Registration
- Dashboard
- Service Request Management
- Request Tracking
- Service Request
- Service Tracking
- Reports

13.3 Business Logic Layer

This layer contains the main application functionality.

Major components include:

Authentication Module

Handles login, registration, and authorization.

Service Request Module

Validates request details, categories, locations, priorities, and current status.

Assignment Module

Assigns requests to technicians and tracks work progress.

Notification Module

Stores and manages service updates and notifications.

Reporting Module

Generates service summaries, request statistics, and workload reports.

13.4 Data Layer

The data layer manages persistent information.

Example collections/tables:

Users

ServiceRequests

Categories

Technicians

StatusHistory

Notifications

ServiceReports

14. Architecture Component Interaction

The interaction between the components is:

User

|

v

Frontend

|

| REST API

v

Authentication Controller

|

+-----+

|

|

v

v

Request Service

Assignment Service

|

|

v

v

Request Repository

Technician Repository

|

|

+-----+-----+

|

v

Database

For example, when the user submits a service request:

User

|

v

Service Request Page

|

v

Service Request API

|

v

Request Processing Service

|

+--> User Data

|

+--> Request Details

|

+--> Priority and Category

|

v

Service Request Result

|

v

Dashboard

15. Quality Attribute Analysis

Performance

The backend should process service requests efficiently and database queries should be optimized.

Security

Authentication, authorization, password hashing, input validation, and secure environment variables should be implemented. Scalability

The stateless backend can be scaled horizontally by creating additional application containers.

Maintainability

Separating controllers, services, models, and routes improves maintainability.

Reliability

Validation and automated testing reduce request-processing and data-management errors.

Portability

Docker allows the same application environment to run on different machines.

Usability

The dashboard should provide simple navigation and understandable service information.

16. Architectural Justification

Three-tier architecture is suitable for the Smart Campus Maintenance and Service Request Management System because it separates responsibilities between different parts of the application.

The presentation layer focuses on user interaction, the business layer handles service requests and assignment workflows, and the data layer manages persistent information.

This separation provides:

- Better maintainability
- Easier testing
- Improved security
- Independent modification of components
- Better scalability
- Clear separation of responsibilities

Docker further improves the architecture by isolating the backend and database services into separate containers.

Git provides controlled source-code management and enables developers to work collaboratively through branches and meaningful commits.

Therefore, combining Three-Tier Architecture + Git + Docker provides a suitable technical foundation for developing and deploying the Smart Campus Maintenance and Service Request Management System.

17. Conclusion

The Smart Campus Maintenance and Service Request Management System provides an organized approach to reporting and resolving campus service issues. The system allows users to submit requests, select categories and priorities, track progress, and receive service updates.

Git provides effective source-code version control, branching, collaboration, rollback, and change tracking. The selected branching strategy separates stable code from ongoing development and service feature implementation.

Docker provides a consistent and portable application environment. Docker Compose allows the backend and database services to be configured and executed together while Docker volumes provide persistent service data storage.

The selected three-tier architecture separates presentation, business logic, and data management. This improves maintainability, scalability, security, and testing.

Overall, the combination of Git, Docker, Docker Compose, and three-tier architecture provides an organized, portable, maintainable, and scalable solution for the proposed Smart Campus Maintenance and Service Request Management System.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Project Design(20%)	Clearly explains the problem statement, objectives, requirements, expected outcomes, and well-organized project structure with proper justification.	Good analysis and project structure with minor omissions.	Basic analysis and repository structure with limited justification.	Incomplete or unclear analysis and project organization.
Git Repository & Branching Strategy(20%)	Repository initialized correctly with appropriate branching strategy, clear workflow, and proper repository organization.	Repository and branching strategy are mostly correct.	Basic Git setup with limited branching implementation.	Incorrect or incomplete Git repository and branching strategy.
Version Control Workflow & Commit History(10%)	Demonstrates meaningful commits, branching, merging, synchronization, conflict resolution, and professional commit	Most Git operations demonstrated correctly with good commit history.	Limited Git workflow and commit practices.	Poor workflow and inadequate commit history.

	messages with clear history.			
Docker Implementation(20%)	Well-designed Docker environment, Dockerfile, and Docker Compose configuration with clear explanation of services, networking, volumes, and dependencies.	Functional Docker implementation with minor issues.	Basic Docker implementation with limited explanation.	Incomplete or incorrect Docker implementation.
Deployment, Testing & Results(15%)	Application successfully deployed and tested with appropriate screenshots and evidence of successful execution.	Deployment completed with minor issues.	Partial deployment and testing demonstrated.	Deployment unsuccessful or insufficient evidence.
Documentation, Challenges & Future Enhancements(15%)	Professional report (10–15 pages) with diagrams, screenshots, references, discussion of benefits, challenges, and future improvements.	Good documentation with minor omissions.	Basic documentation with limited discussion.	Poor documentation and insufficient analysis.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25